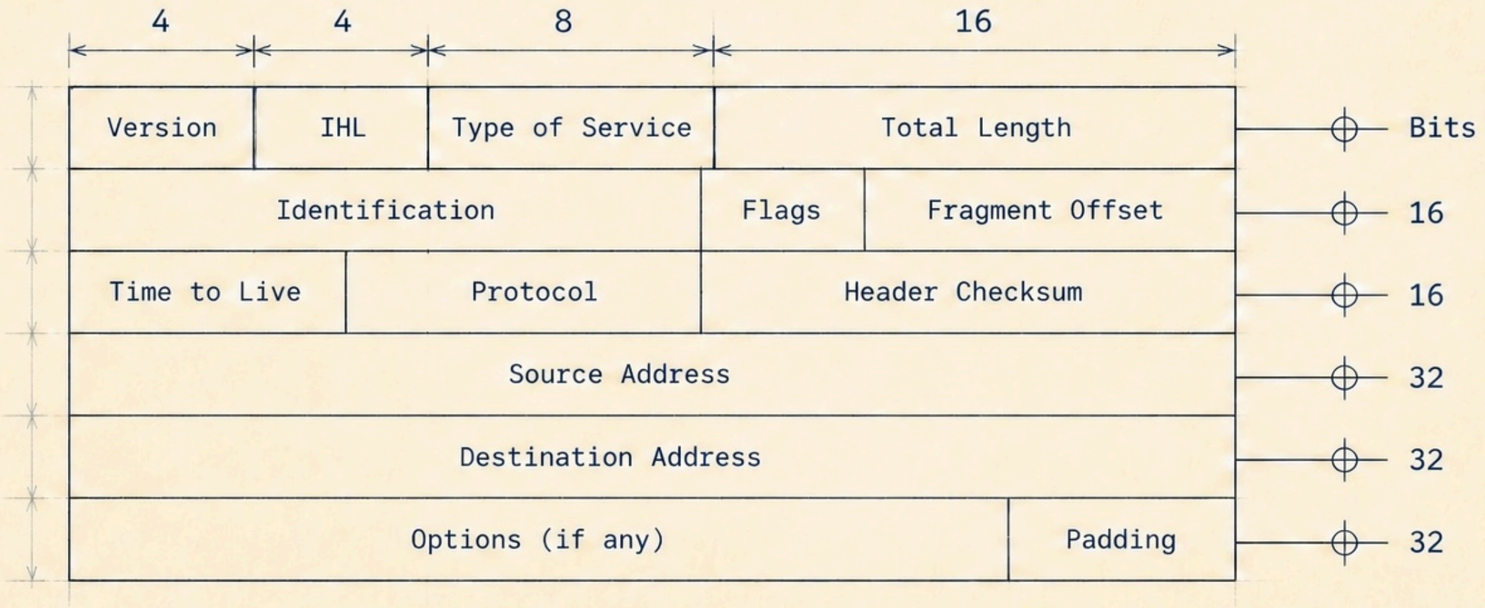
By Nabajyoti Medhi

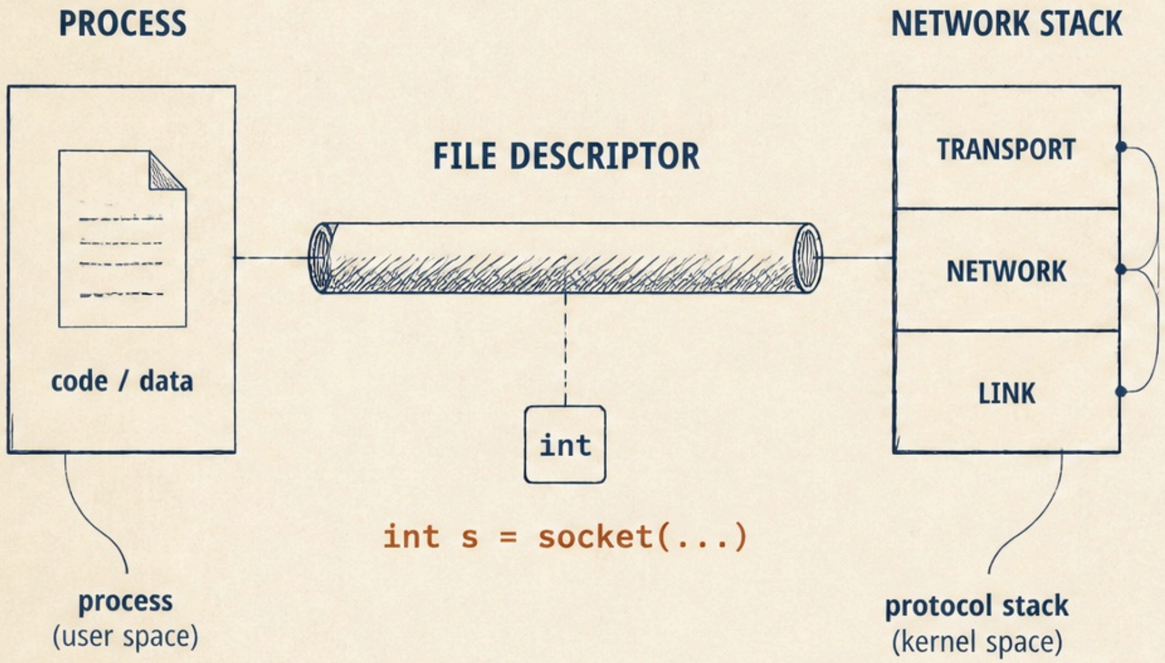
Raw Sockets in C

Crafting, sniffing, and steering packets from userspace

IPv4 Header
(20 bytes min)

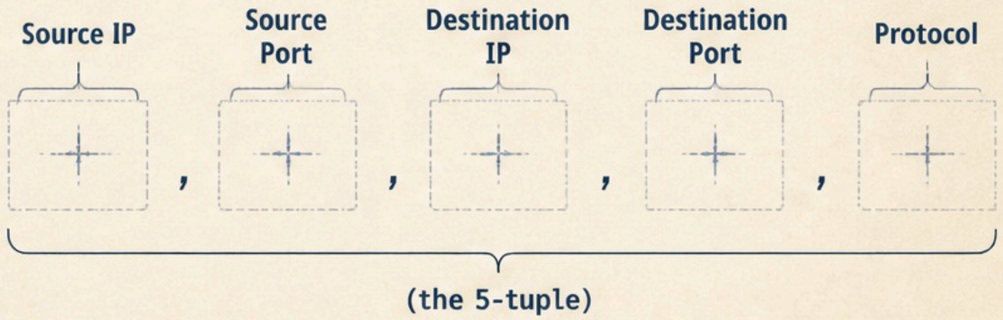


A socket is just a file descriptor that talks to a network stack

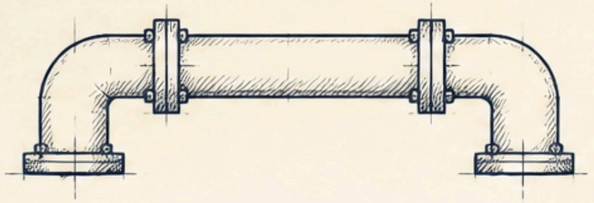


1. a file descriptor (int)
2. lives in the process file table
3. talks to a protocol stack

SOCKET IDENTIFIED BY THE FIVE-TUPLE



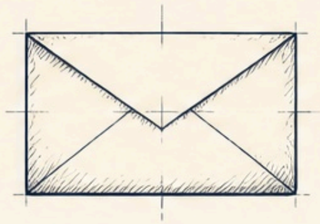
Three flavors, one socket() call



SOCK_STREAM

Protocol hint:	TCP
Connection:	Connection-oriented
You supply:	None. Kernel manages reliability, ordering, flow control.
Kernel supplies:	Reliability, ordering, flow control, congestion control.

```
socket(AF_INET, SOCK_STREAM,
       IPPROTO_TCP)
```



SOCK_DGRAM

Protocol hint:	UDP
Connection:	Connectionless
You supply:	Message boundaries (datagrams) and destinations.
Kernel supplies:	Checksums, routing, best-effort delivery.

```
socket(AF_INET, SOCK_DGRAM,
       IPRROTO_UDP)
```



SOCK_RAW

Protocol hint:	Typically ICMP, IGMP, OSPF or custom
Connection:	Connectionless
You supply:	IP header (often) and protocol-specific payload.
Kernel supplies:	Routing and (often) checksums.

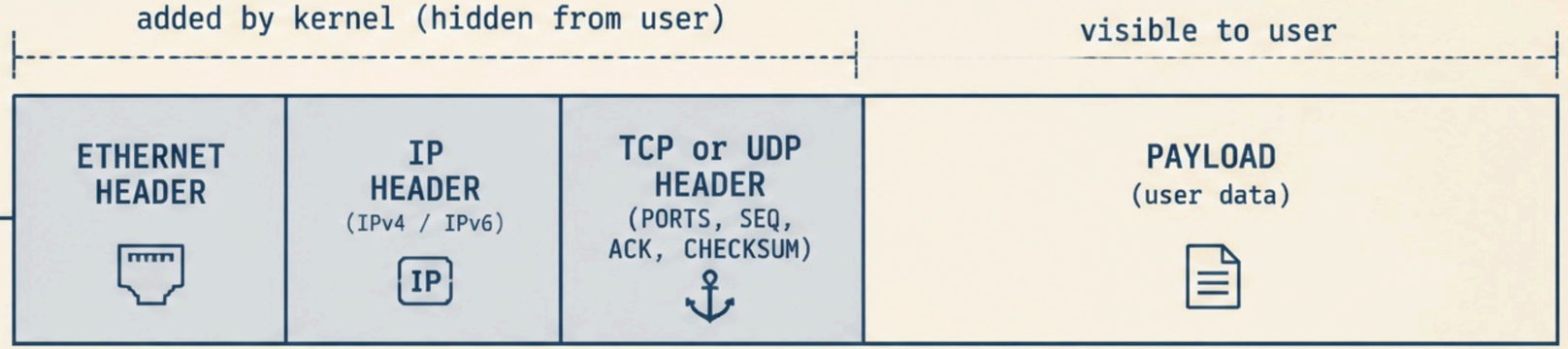
```
socket(AF_INET, SOCK_RAW,
       IPROTO_ICMP)
```

Requires privilege on every major OS

What regular sockets can't reach

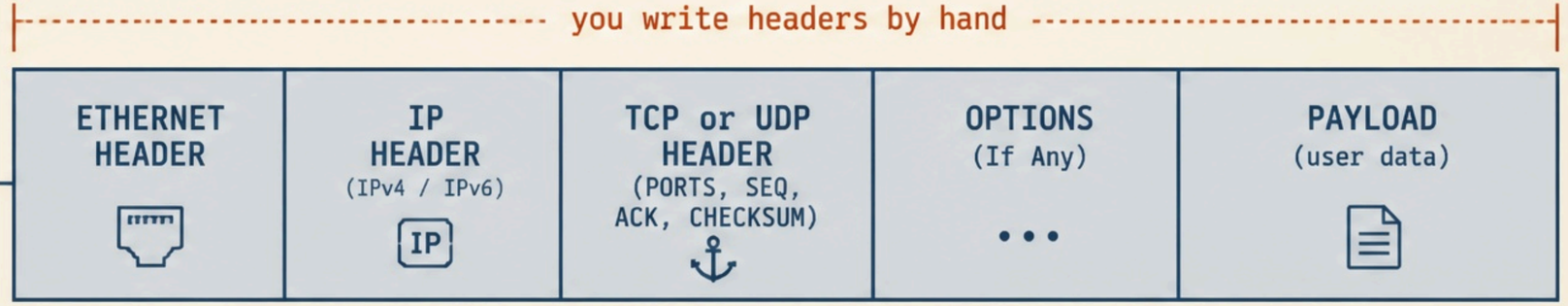
REGULAR SOCKET

stack adds Ethernet / IP / TCP or UDP headers – user only fills payload



RAW SOCKET

user can build (almost) everything – headers and payload under your control



SOCK_STREAM

reliable, connection-oriented (TCP)

SOCK_DGRAM

connectionless (UDP)

SOCK_RAW

you craft the packet headers

Open one. Three parameters, one decision.

```c
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>    // for IPPROTO_*

int main(void) {

    int fd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);

    if (fd < 0) {
        perror("socket");
        return 1;
    }

    return 0;
}
```

AF_INET or AF_PACKET

SOCK_RAW

protocol hint
(e.g. IPPROTO_ICMP /
TCP / UDP / custom)

on Linux the protocol
need not be a registered one

- returns a file descriptor
- no listen() / no accept()
- reads and writes packets, not streams

Linux is permissive, Windows is fenced

Linux

- AF_INET and AF_PACKET both supported
- IP_HDRINCL works as expected
- can bind to interface names
- raw TCP / UDP / ICMP straightforward
- CAP_NET_RAW required

IP_HDRINCL behavior

Windows

- Winsock only (winsock2.h / ws2_32.lib)
- WSAStartup / WSACleanup required
- IP_HDRINCL disabled since XP SP2 – kernel always builds IP header
- AF_PACKET needs Npcap / WinPcap
- Administrator token required

Raw sockets need clearance



Raw sockets let you craft, sniff, and steer packets — so the OS requires proof you're trusted to do so.

Linux

- CAP_NET_RAW capability required
- usually means running as root
- containers often drop this capability
- unprivileged ping works only on a few recent kernels

Windows

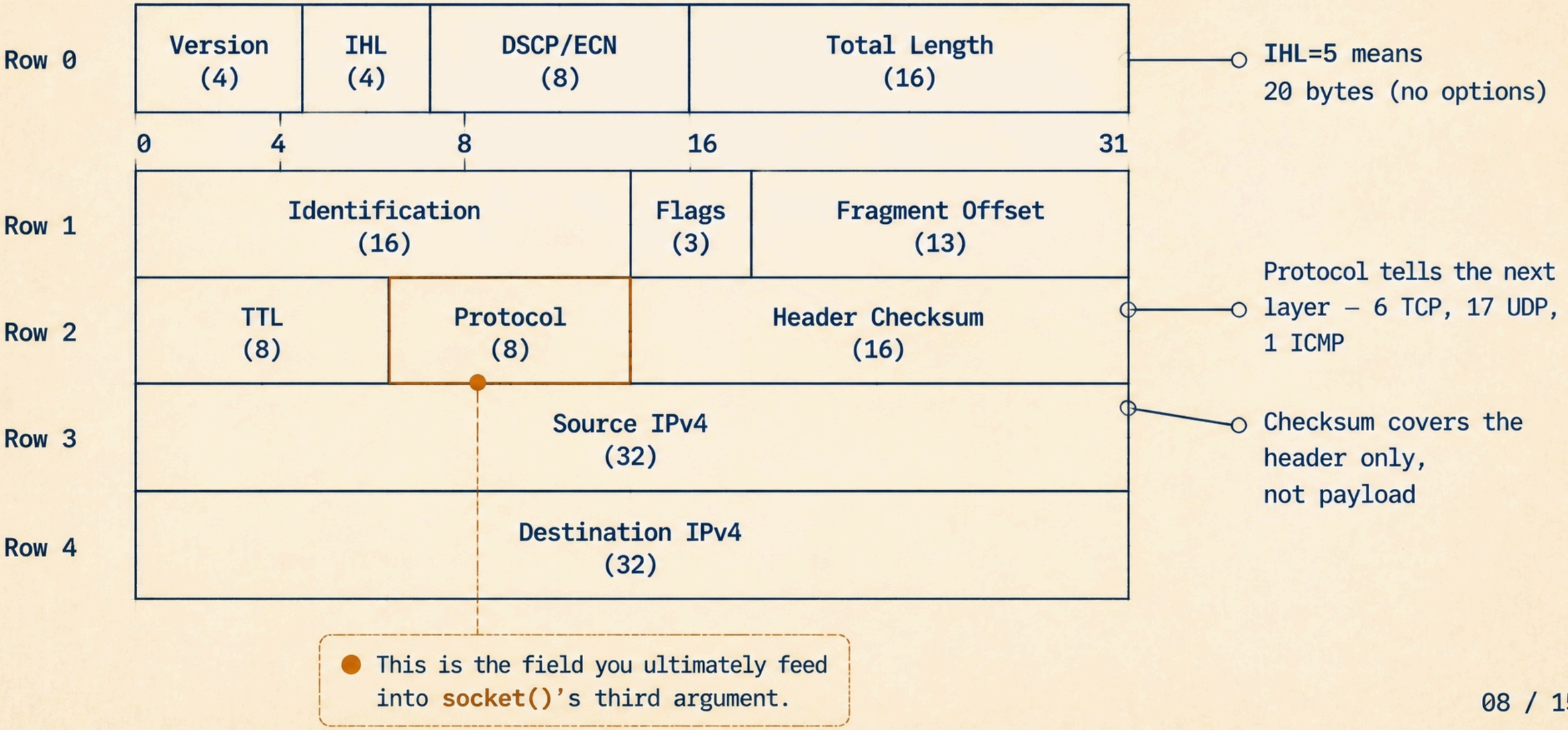
- member of Administrators group
- service accounts run from the SCM
- UAC may prompt at launch

macOS

- sudo / root for one-off tooling
- signed apps need entiilements
- System Integrity Protection does not gate socket creation explicitly

```
if (getuid()) != 0) {
      fprintf(stderr, "need root\n");
      return 1;
}
```

The IPv4 header, 20 bytes laid out



Forge the IP header yourself

`IP_HDRINCL = 0` → kernel builds the header

`IP_HDRINCL = 1` → you build the header

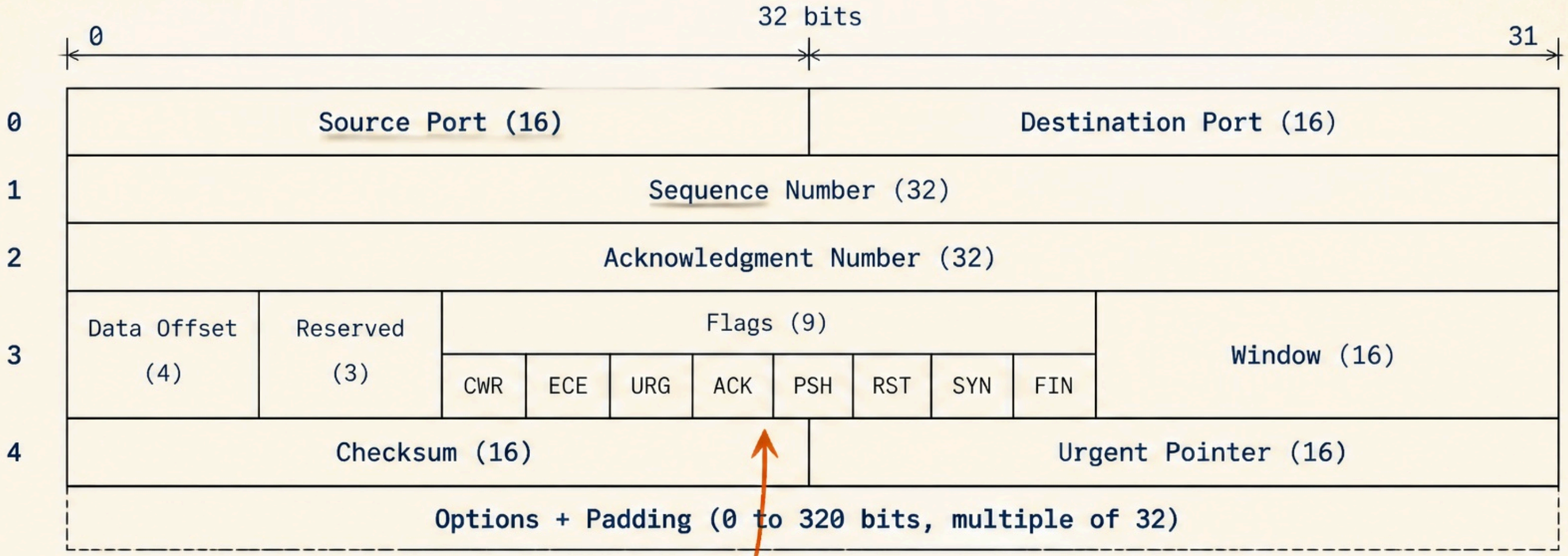
```
int one = 1;
setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one));

struct iphdr ip = {0};
ip.version   = 4;          // IPv4
ip.ihl       = 5;          // 5 * 4 = 20 bytes (no options)
ip.tot_len   = htons(sizeof(struct iphdr) + payload_len);
ip.id        = htons(0x1a2b);
ip.ttl       = 64;
ip.protocol  = IPPROTO_TCP;
ip.saddr     = inet_addr("192.0.2.11");
ip.daddr     = inet_addr("198.51.100.2");
ip.check     = 0;              // let kernel compute checksum
ip.check     = ip_cheksum(&ip, ip.ihl * 4); // optional: compute yourself
```

remember: total, not just header

- set before the first send
- a zero value inside ip.check lets the kernel recompute for you
- put socket in IP_HDRINCL with IPPROTO_RAW for maximum control
- bind() before sendto() if you want to specify a source

TCP, 20 bytes of stateful conversation



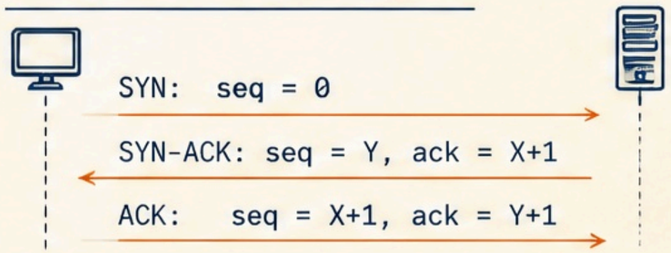
The Nine Control Flags

CWR	Congestion Window Reduced (ECN)
ECE	ECN-Echo
URG	Urgent
ACK	Acknowledgment
PSH	Push
RST	Reset
SYN	Synchronize
FIN	Finish

★ **Most used flags in TCP:**

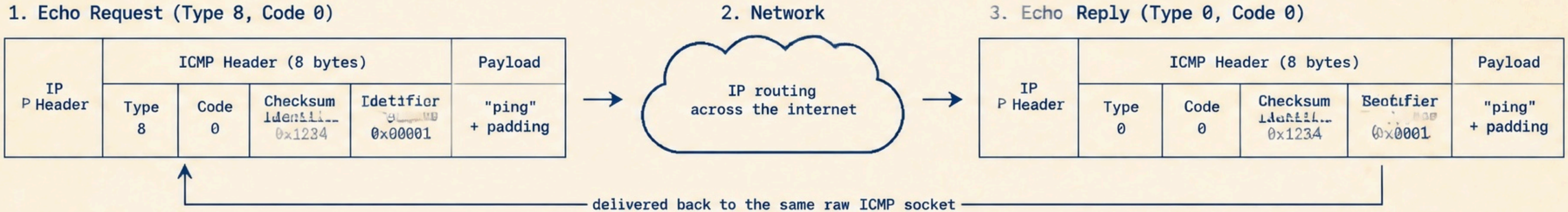
- **SYN** – Starts a connection
- **ACK** – Confirms received data
- **FIN** – Gracefully closes a connection

A tiny handshake (3 packets)



X = initial sequence number from client
Y = initial sequence number from server

ICMP echo from scratch




```c
int fd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);

struct icmphdr *icp;
char sendbuf[64];
memset(sendbuf, 0, sizeof(sendbuf));

icp = (struct icmphdr *)sendbuf;
icp->type = ICMP_ECHO;
icp->code = 0;
icp->un.echo.id = htons((uint16_t)getpid());
icp->un.echo.sequence = htons((uint16_t)seq++);

const char *msg = "ping";
memcpy(sendbuf + sizeof(struct icmphdr), msg, strlen(msg));

size_t pkt_len = sizeof(struct icmphdr) + strlen(msg);
icp->checksum = 0;
icp->checksum = in_cksum((uint16_t *)sendbuf, pkt_len);

struct sockaddr_in dst;
memset(&dst, 0, sizeof(dst));
dst.sin_family = AF_INET;
dst.sin_port = 0;
inet_pton(AF_INET, "203.0.113.1", &dst.sin_addr);

ssize_t n = sendto(fd, sendbuf, pkt_len, 0,
                   (struct sockaddr *)&dst, sizeof(dst));
```

ICMP_ECHO (Type 8)
Code must be 0

Compute 16-bit one's-complement checksum over the entire ICMP message (header + payload)

- the kernel delivers matching echo replies to the same socket
- recvfrom() blocks until reply (or until SO_RCVTIMEO expires)
- time the round-trip yourself to get latency

Checksum math, one line at a time

```
unsigned short checksum(void *b, int len)
{
      unsigned long sum = 0;                          // 32-bit accumulator
      unsigned short *p = (unsigned short *)b;        // 16-bit words
      int n = len;

      while (n > 1) {
          sum += *p++;                                // add next 16-bit word
          if (sum & 0x10000)                          // if carry out of bit 15
              sum = (sum & 0xFFFF) + 1;               // fold carry into low 16
          n -= 2;
      }

      if (n > 0) {                                    // odd trailing byte
          unsigned short last = *(unsigned char *)p << 8;// high-order byte
          sum += last;
          if (sum & 0x10000)
              sum = (sum & 0xFFFF) + 1;
      }

final
result
returned
      return ~sum;                                    // one's complement (bitwise NOT)
}
```

Worked example (12 bytes)

1) Example buffer (hex)

00	01	F2	03	04	05	06	07	08	09	0A	0B

2) 16-bit words (big-endian) and running sum

word		added		sum (hex)	
0x0001	+	0x0001	→	0x0001	
0xF203	+	0xF203	→	0xF204	
0x0405	+	0x0405	→	0xF609	
0x0607	+	0x0607	→	0xFC10	
0x0809	+	0x0809	→	0x10419	(carry)
0x0A0B	+	0x0A0B	→	0x10E24	(carry)

3) Fold carries into 16 bits

0x10E24 → (0x0E24 + 1) = 0x0E25

4) One's complement (bitwise NOT)

~0x0E25 = **0xF1DA** ← final checksum

- used by IPv4 header, ICMP, and UDP
- TCP and UDP first prepend a 12-byte pseudo-header (src, dst, zero, protocol, length) before summing
- a correctly received packet should produce 0 when you re-checksum it header and all

Reading what isn't yours

AF_PACKET and ETH_P_ALL let you see link-layer frames too.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

#define BUFSZ 65536

int main(void) {
    int fd = socket(AF_INET, SOCK_RAW, IPROTO_TCP);
    if (fd < 0) { perror("socket"); return 1; }
    unsigned char buf[BUFSZ];

    for (;;) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n < 0) { if (errno == EINTR) continue; perror("recv"); break; }
        if (n < (ssize_t)sizeof(struct iphdr)) continue;
        struct iphdr *ip = (struct iphdr *)buf;
        if (ip->version != 4) continue;        // IPv4 only
        if (n < (ssize_t)(ip->ihl * 4)) continue;
        unsigned char *l4 = buf + ip->ihl * 4;  // advance to next header
        uint16_t iplen = ntohs(ip->tot_len);
        if (iplen > (uint16_t)n) continue;
        struct in_addr src = ip->saddr;
        struct in_addr dst = ip->daddr;
        printf("%s -> %s len=%u proto=TCP\n",
               inet_ntoa(src), inet_ntoa(dst), (unsigned)iplen);
    }
    close(fd);
    return 0;
}
```

the kernel hands you the IP header along with payload - peel to follow as needed.

- poll() for non-blocking reads
- one thread per protocol hint is common
- setsockopt(SO_RCVBUF) to grow the receive buffer on busy hosts

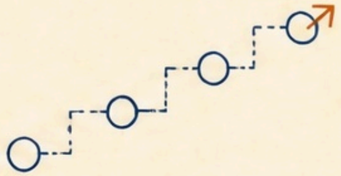
Where this is useful

ping



- Send and receive ICMP Echo with full control.

traceroute



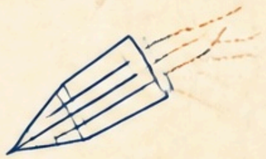
- Send probes with increasing TTL and read Time Exceeded.

path MTU discovery



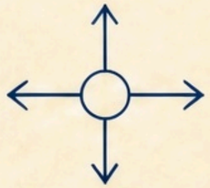
- Probe with DF set and adjust to avoid fragmentation.

custom probes



- Craft and send custom IP/TCP/UDP/ICMP packets.

routing daemons (Zebra / Quagga)



- Receive protocol packets (RIP, OSPF, BGP) and send updates.

IDS / passive monitors



- Capture and inspect packets without kernel filtering.

packet generators (hping3, nmap, scapy)



- Generate high-rate or crafted traffic for testing.

ICMP-based diagnosis beyond stock ping



- Use other ICMP types/codes not exposed by ping.

Read the man pages.
Write small tools.
Use responsibly.

man 7 raw
man 7 packet
RFC 791
RFC 793
RFC 768 -- start here

- write the smallest tool that solves your question, comment generously

- do not scan, spoof, or flood networks you do not own or have written permission to test

